

Neural Networks and Deep Learning

HW #2 Report

Tiny Shakespeare Memory Duel

Elman Network vs. Jordan Network for Character-Level Language Modeling

Student Name	Mana Peiravian
Student ID	40464250

1. Executive Summary

This report compares two recurrent neural network architectures for character-level next-character prediction on the provided Tiny Shakespeare text corpus. The Elman network uses hidden-state feedback, while the Jordan network uses feedback from the previous output probability distribution. Both models were trained under the same preprocessing, sequence length, optimizer, batch size, and evaluation pipeline.

In this experiment, the full Tiny Shakespeare dataset was used. Therefore, `DATA_FRACTION = 1.00`, corresponding to 100% of the available text. The dataset was treated as a character-level corpus, and sequences of length 100 were used for next-character prediction.

2. Dataset and Preprocessing

The dataset was treated as a character-level corpus. No word tokenization or normalization was applied, so capitalization, punctuation, newlines, and speaker labels were preserved.

Preprocessing steps:

- Read the raw Tiny Shakespeare text file.
- Use the whole dataset for the submitted experiment.
- Build a sorted character vocabulary and integer mappings.
- Encode the corpus as integer IDs.
- Split the selected subset into 90% training and 10% validation.
- Create input/target pairs with sequence length 100; the target sequence is shifted one character ahead.

Item	Value
Original characters	1,115,394
Characters used	1,115,394
Vocabulary size	65
Training sequences	10,038
Validation sequences	1,115
Sequence length	100
Batch size	128
Epochs	20
Device used	cpu

3. Model Architectures

3.1 Elman Network

The Elman model keeps a hidden memory vector and feeds it back into the recurrent computation at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b), \quad \text{logits}_t = W_o h_t + c$$

Because the feedback signal is the hidden state, the model can store a compressed representation of preceding characters.

3.2 Jordan Network

The Jordan model feeds the previous output distribution back into the recurrent computation:

$$h_t = \tanh(W_x x_t + W_c y_{t-1} + b), \quad y_t = \text{softmax}(W_o h_t + c)$$

Here y_{t-1} is the predicted probability distribution over the vocabulary. This makes the model condition on its previous character-level belief rather than on a learned hidden state.

3.3 Shared Hyperparameters

Hyperparameter	Value
Embedding dimension	128
Hidden size	256
Optimizer	Adam
Learning rate	0.004
Gradient clipping	1.0
Loss	Cross-entropy
Sampling temperatures	0.4, 0.8, 1.2
Seed text	'ROMEO:\n'

Both models used an embedding dimension of 128 and a hidden size of 256. This larger configuration was selected to give the recurrent networks enough capacity to learn character-level patterns from the full Tiny Shakespeare dataset.

Both models were trained for 20 epochs using the Adam optimizer with learning rate 0.004. Gradient clipping with maximum norm 1.0 was applied to reduce the risk of exploding gradients. The same batch size, sequence length, embedding dimension, hidden size, and optimizer settings were used for both models to ensure a fair comparison.

The learning rate 0.004 was used and the loss curves remained stable and no divergence was observed during training.

The Elman model used PyTorch’s built-in nn.RNN layer with tanh activation, corresponding to hidden-state feedback.

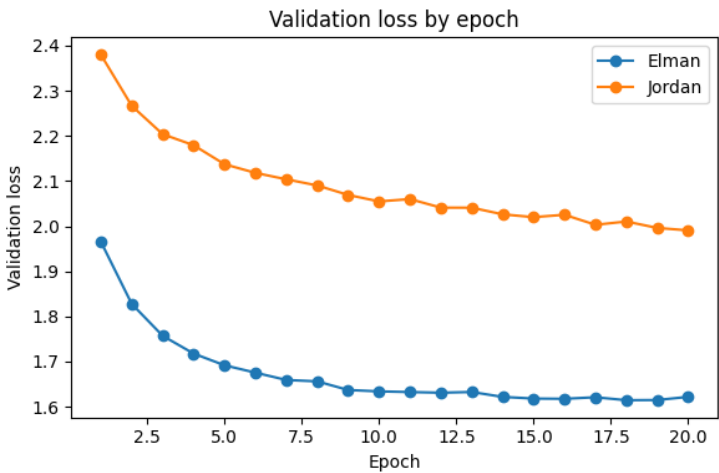
The Jordan model was implemented manually because PyTorch does not provide a built-in Jordan recurrent layer. In this model, the previous output probability distribution was used as the recurrent feedback signal.

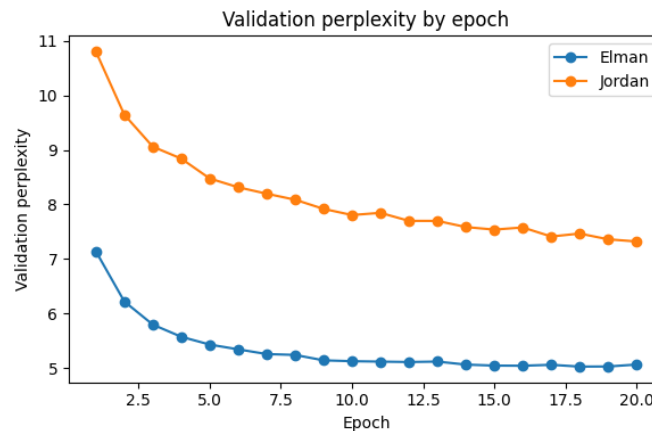
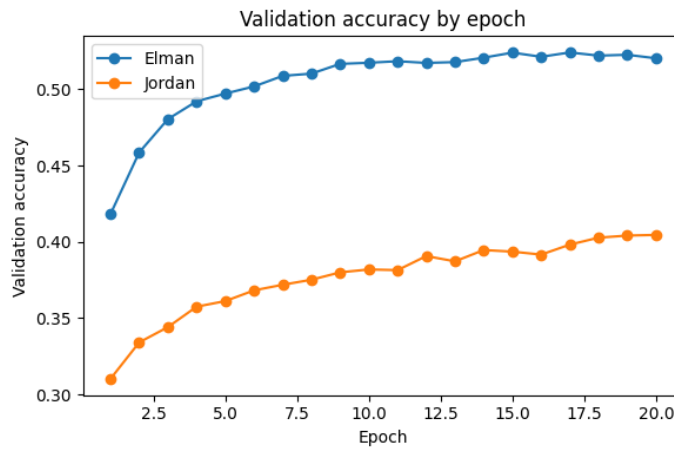
4. Training and Validation Results

Evaluation used cross-entropy loss, character-level accuracy, and perplexity. Perplexity was computed as $\exp(\text{cross_entropy_loss})$. Lower loss and lower perplexity are better; higher accuracy is better.

Model	Epoch	Train loss	Train acc	Train PPL	Val loss	Val acc	Val PPL
Elman	8	1.39102	57.2792%	4.018947	1.62188	52.0395%	5.062601
Jordan	8	1.88474	43.1289%	6.584644	1.99070	40.4583%	7.320660

Main numerical finding: in this submitted CPU run, the Elman model achieved the best validation metrics. The Jordan model was stable but finished slightly behind. The Elman model reached its highest validation accuracy of 52.42% at epoch 17, while its final epoch-20 validation accuracy was 52.04%.





5. Text Generation and Temperature Study

Both models generated samples from the same seed text. Temperature scaled the logits before sampling: lower values concentrate probability mass on high-confidence characters, while higher values increase randomness.

Temperature = 0.4

Model	Generated sample
Elman	ROMEO: I present of this scorn the presence with his intelligence to the thoughts that I will passions and by the sun and some princely men are well thou shalt not that I will thou shall not I shall be a presence to the need. WARWICK: I would have not thou art thou cannot in this love safe of the house and leave your hands of things of this earth shall say you will not a should in the noble grace of the points of this hand; and what I should thou hast thou not shame of my single of this proportion tha
Jordan	ROMEO: And be the me shand his best of the they that the hall so may be be the dee froughter of the not my houggly to have and hath the have the the in the my be will the come the it the the have my and the lord, and the be be the the were stise of the shanger and have the my lork the like the have be the have seath so so my not

Model	Generated sample
	it so be the here be many man sined the the the will be worst, where ease he here the mene the bene sore the be of to love and there the so the come to to his cough my ban the

Temperature = 0.8

Model	Generated sample
Elman	ROMEO: I say to the vanyingbree not a means, or in Paulinate To thee, sir, that think thou shalt fast. BUCKINGHAM: Who may you'll. ROMEO: And supplent with a mother sunse conscience That life, shall misers, and her tower! Thy will not think my way to you. CLIFFORD: What I would I will. POMPEY: The advench of a while! MONTAGUE: By the privometh affords of plain to their censure hast wish'd death-side! COMINIUS: It is that Grace and world; Yet would be the long art thou had you laster. ROMEO: Noth
Jordan	ROMEO: Then blous, it bo thoung call in be of boy, Welery. Is, there, I's bemnat or and there to say sper of Muted, Was untakey but peping besting do cansting: Whee man come. O, your the then have his ro preat the in then in me my word, and doce have no their. QUS: Hough to doet-have but bed? Housean that that the housanes, I for be touoring meadds may withe's bust you how, being wheargue, were the frost wall lose your tevine. KINGENTES: My beer the cath and than hat exishting bagost, Sperry the beizv

Temperature = 1.2

Model	Generated sample
Elman	ROMEO: An Anto them. Edward, what shadugtize you, I myself; Thou blood from time that deadly duttle:

Model	Generated sample
	Consing by queecas brin? Come the words. In't youn. QUEEN MARGARET: O may resoln have you much? My speck patteror's tenty's power'd: or brought good. KING RICHARL WARD GREY: Howh'T A can are, Aumeety to Hownd him to the tekso's soleges, Go eyec, Hath mighty of Murfural appair, And;-I shall. Wourse lie: looking The law. COMINIUS: For thy against your choopd! you speak; That 'londs, though I may be Hen
Jordan	ROMEO: Cheap. MERY: I my Hill; you, And comed. You: this be that nome, Them raw; For sust. JO:'s He arn 'They crimie and nobish. BY: Autakich jow, somplad! So lopyy rukn: Wele; with? So have entagow devomey: Te pere, Seath. 'Spith, welthimb. ROMILIO: som'st, to sroy the'd yeay off? Anslorks, thill mys, BE: G\$resteavus her-tits siss cithe, That noven utin was layine. ANTES: Gruke be donfing davee till in youss'st and frougshaier Wand; dong, Thing. Beatily spurples! Novinot? King way, wire. 3ICLUM

6. Comparison and Discussion

6.1 Convergence Speed

The Elman network converged much faster than the Jordan network under the same experimental settings. At epoch 1, the Elman model reached a validation loss of 1.9656 and validation accuracy of 41.82%, while the Jordan model reached a validation loss of 2.3800 and validation accuracy of 31.02%. This already shows that the Elman architecture learned useful character-level patterns earlier in training.

The performance gap remained throughout training. By epoch 10, Elman reached validation loss 1.6339 and validation accuracy 51.75%, while Jordan reached validation loss 2.0548 and validation accuracy 38.19%. At epoch 20, Elman finished with validation loss 1.6219 and validation accuracy 52.04%, while Jordan finished with validation loss 1.9907 and validation accuracy 40.46%.

This indicates that hidden-state feedback provided a more useful memory mechanism than output-distribution feedback. In the Elman model, the previous hidden state is a flexible learned representation of past context. In the Jordan model, the recurrent feedback comes from the previous output distribution, which is a narrower signal because it mostly represents the model's belief about the next character.

6.2 Stability

Both models trained stably across 20 epochs. There was no sign of exploding loss or numerical divergence. Gradient clipping with a maximum norm of 1.0 helped stabilize the recurrent training process.

The Elman model showed fast improvement in the early epochs and then gradually plateaued. Its validation loss decreased from 1.9656 at epoch 1 to 1.6219 at epoch 20. Its validation accuracy increased from 41.82% to 52.04%. The best validation accuracy for Elman was 52.42% at epoch 17, showing that the model had mostly converged by the later epochs.

The Jordan model also improved steadily but much more slowly. Its validation loss decreased from 2.3800 at epoch 1 to 1.9907 at epoch 20, and its validation accuracy increased from 31.02% to 40.46%. Unlike Elman, Jordan was still improving slowly near the end of training, but it remained far behind Elman.

There is no strong evidence of severe overfitting. For Elman, the final training accuracy was 57.28% and validation accuracy was 52.04%, giving a moderate gap of about 5.24 percentage points. For Jordan, the training accuracy was 43.13% and validation accuracy was 40.46%, giving a smaller gap of about 2.67 percentage points. These gaps are reasonable for this task and do not indicate serious memorization.

6.3 Validation Performance

The Elman model achieved clearly better final validation performance across all metrics. At epoch 20, Elman obtained validation loss 1.6219, validation accuracy 52.04%, and validation perplexity 5.0626. The Jordan model obtained validation loss 1.9907, validation accuracy 40.46%, and validation perplexity 7.3207.

The lower validation loss and perplexity of the Elman model show that it assigned higher probability to the correct next characters. Its higher validation accuracy also shows that its top prediction was correct more often. Compared with Jordan, Elman improved validation accuracy by about 11.58 percentage points and reduced validation perplexity from 7.3207 to 5.0626.

The training metrics show the same pattern. Elman reached training loss 1.3910, training accuracy 57.28%, and training perplexity 4.0189. Jordan reached training loss 1.8847, training accuracy 43.13%, and training perplexity 6.5846. Since Elman performs better on both training and validation data, the result suggests that the Elman architecture has a stronger learning capacity for this character-level task rather than simply overfitting.

6.4 Generated Text Quality

The generated text samples support the quantitative results. The Elman model produced more recognizable Shakespeare-like structure than the Jordan model. It learned common word patterns, punctuation, line breaks, and speaker-name formatting more effectively.

At temperature 0.4, the Elman output was conservative but relatively structured. It produced repeated but readable fragments such as “I present,” “the presence,” “I shall,” and “WARWICK:”. The text was still not fully coherent, but it showed recognizable phrase structure and play-script formatting. The Jordan output at the same temperature was much more repetitive, heavily overusing words such as “the,” “be,” “have,” and “my.” This suggests that Jordan relied more strongly on frequent local patterns and had difficulty forming stable longer phrases.

At temperature 0.8, the Elman model produced more diverse and more dramatic text. It generated multiple speaker-style labels such as “BUCKINGHAM:”, “ROMEO:”, “CLIFFORD:”, “POMPEY:”, “MONTAGUE:”, and “COMINIUS:”. Although many words were still misspelled or invented, the structure looked more like a Shakespeare play. The Jordan model also generated some speaker-like formatting, such as “QUS:” and “KINGENTES:”, but the output was less coherent and contained more malformed words.

At temperature 1.2, both models became noisier because the sampling process became more random. The Elman model still produced some recognizable speaker formatting, including “QUEEN MARGARET:” and “COMINIUS:”, but many words became distorted. The Jordan model became much more unstable, producing many short broken fragments, unusual symbols, and malformed speaker names such as

“ROMILIO:” and “3ICLUM”. This shows that Jordan was more sensitive to high-temperature sampling.

Overall, Elman generated better text than Jordan. It was not perfectly grammatical, but it had stronger structure, better formatting, and more recognizable Shakespeare-like fragments. Jordan learned basic local character patterns, but its generated text was more repetitive at low temperature and more unstable at high temperature. This agrees with the final validation metrics, where Elman had lower loss, higher accuracy, and lower perplexity.

6.5 Effect of Temperature

The temperature experiments showed the expected stability-diversity trade-off. At temperature 0.4, both models generated safer but more repetitive text. The Elman model remained relatively readable, while the Jordan model repeated common words too often. This happened because low temperature increases the probability of already likely characters, making the model more conservative.

At temperature 0.8, the generated text became more varied. This temperature provided the best balance between stability and creativity. The Elman model produced the strongest outputs at this setting because it generated several speaker labels and more varied phrase structures while still preserving some play-like formatting. The Jordan model also became more diverse, but its words were more distorted.

At temperature 1.2, both models became more random. The generated text had more variety, but coherence decreased. The Elman model still preserved some dramatic formatting, but the Jordan model became highly unstable, producing broken words, unusual symbols, and inconsistent structure. Therefore, temperature 0.8 was the best overall choice for generation quality in this experiment.

7. Key Implementation Points

- Character-level modeling was used instead of word-level modeling. Each unique character in the Tiny Shakespeare corpus was mapped to an integer index, and the models learned to predict the next character at each time step.
- The full Tiny Shakespeare dataset was used. The final experiment used `DATA_FRACTION = 1.00`, corresponding to 100% of the available text.
- The sequence length was set to 100. This means that each input sample contained 100 consecutive characters, and the target sequence was the same text shifted by one character.
- Non-overlapping fixed-length sequences were used. In other words, the stride was equal to the sequence length. With `SEQ_LEN = 100`, the first sequence used characters 0–99, the second used characters 100–199, and so on. This kept the number of training samples manageable and reduced redundant computation.

- Overlapping sequences were not used in the final experiment. If a smaller stride, such as 1, had been used, the training data would contain many highly similar samples, such as characters 0–99, 1–100, 2–101, and so on. This could provide more training examples and may improve accuracy, but it would greatly increase runtime and memory usage. It could also make validation results misleading if overlapping sequences were created before the train/validation split, because nearly identical sequences could appear in both sets.
- To avoid data leakage, the raw text was split into training and validation portions before sequence generation. This ensured that validation sequences came from a separate part of the corpus rather than from almost identical overlapping windows.
- The same train/validation split, sequence length, batch size, embedding dimension, hidden size, optimizer, learning rate, and number of epochs were used for both architectures. This made the Elman and Jordan comparison fair.
- Both models used an embedding dimension of 128 and a hidden size of 256. The larger embedding allowed richer character representations, while the larger hidden state increased the recurrent memory capacity.
- Cross-entropy loss was applied at every time step. Since this is a next-character prediction task, the model produced a probability distribution over the vocabulary for each character position in the sequence.
- Perplexity was computed as $\exp(\text{average cross-entropy})$. Lower perplexity means that the model is less uncertain about the correct next character.
- Gradient clipping with a maximum norm of 1.0 was used during training. This helped reduce the risk of exploding gradients, which can occur in recurrent neural networks.
- Text generation used the same seed text, “ROMEO:\n”, for both models. The seed was first passed through the model to warm up the recurrent state, and then the model generated one new character at a time.
- Temperature sampling was used during generation. The logits were divided by the temperature before applying softmax. Lower temperature, such as 0.4, made the output more conservative and repetitive. Medium temperature, such as 0.8, gave the best balance between stability and variety. Higher temperature, such as 1.2, increased randomness and creativity but also produced noisier text.

8. Conclusion

Using the full Tiny Shakespeare dataset with sequence length 100, embedding dimension 128, hidden size 256, and 20 training epochs significantly improved the results compared with the smaller preliminary configuration. The Elman network achieved the best overall performance, with final validation accuracy of 52.04% and validation perplexity of 5.0626. The Jordan network reached final validation accuracy of 40.46% and validation perplexity of 7.3207.

The results show that hidden-state feedback is more effective than output-distribution feedback for this character-level language modeling task. The Elman model converged faster, achieved better

validation performance, and generated more structured Shakespeare-like text. The Jordan model learned some local character patterns and formatting, but it was more repetitive at low temperature and less stable at high temperature.

The temperature experiments showed that low temperature produced conservative and repetitive outputs, medium temperature gave the best balance between readability and diversity, and high temperature produced more creative but noisier text.

Appendix: Core Model Code

The full runnable code is provided in the accompanying notebook. Core architecture code is reproduced below for reference.

```
class ElmanCharRNN(nn.Module):
    """Elman RNN:  $h_t = \tanh(W_x x_t + W_h h_{t-1})$ ."""
    def __init__(self, vocab_size, embed_dim=32, hidden_size=48):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.RNN(embed_dim, hidden_size, batch_first=True, nonlinearity='tanh')
        self.output = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, state=None):
        emb = self.embedding(x)
        hidden_seq, state = self.rnn(emb, state)
        logits = self.output(hidden_seq)
        return logits, state

class JordanCharRNN(nn.Module):
    """Jordan RNN:  $h_t = \tanh(W_x x_t + W_c y_{t-1})$ , where  $y$  is softmax output."""
    def __init__(self, vocab_size, embed_dim=32, hidden_size=48):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.input_to_hidden = nn.Linear(embed_dim, hidden_size)
        self.context_to_hidden = nn.Linear(vocab_size, hidden_size, bias=False)
        self.output = nn.Linear(hidden_size, vocab_size)
        self.vocab_size = vocab_size

    def forward(self, x, context=None):
        batch_size, time_steps = x.shape
        if context is None:
            context = torch.zeros(batch_size, self.vocab_size, device=x.device)

        emb = self.embedding(x)
        logits_over_time = []
        for t in range(time_steps):
            hidden = torch.tanh(self.input_to_hidden(emb[:, t]) + self.context_to_hidden(context))
            logits = self.output(hidden)
            logits_over_time.append(logits)
            context = F.softmax(logits, dim=-1)

        return torch.stack(logits_over_time, dim=1), context
```